



UNIVERSIDAD NACIONAL DE JUJUY

FACULTAD DE INGENIERÍA

Estándares de Codificación

LABORATORIO DE PROGRAMACIÓN

ORIENTADA A OBJETOS I



Ing. Fabiana R. Aragón

A.P.U Sergio A. Gómez

- Abril 2013 -

Contenido

Introducción.....	2
Convenciones de Nombres.....	3
Pautas Generales.....	3
Formato.....	3
Diseño de Clases	3
Indicación de Alcance	4
Comentarios	4
Intellisense Comentarios	4
End-Of-Line Comentarios.....	4
Línea Individual Comentarios.....	4
//TODO: Comentarios.....	5
C-Style Comentarios	5
Capitalización y nomenclatura.....	5
Capitalización	5
Naming.....	6
Programación.....	6
Namespaces.....	6
Clases y estructuras	7
Interfaces.....	7
Constantes.....	8
Las enumeraciones	8
Atributos.....	9
Propiedades	9
Métodos.....	9
Variables y parámetros	10
Controladores de eventos	11
Control de errores	11
Nombres de Controles	12
Namespace	13

Introducción

Este documento contiene las convenciones de codificación y pautas de estilo que aseguren que el código C # será de calidad consistente y superior. Incluye directrices generales de codificación, así como discusiones sobre nombramiento, el formato y los patrones de uso del lenguaje.

Este documento describe reglas y recomendaciones para el desarrollo de aplicaciones y librerías de clases usando el lenguaje C#. La meta es definir líneas guías para enmarcar la consistencia de estilo y de formato y ayudar a los desarrolladores a evadir errores comunes.

A pesar de ser un tema complejo, crear estándares para la programación puede ser de mucha utilidad para desarrolladores dentro de la organización, es por eso que estos estándares deben tender a lo práctico.

Las técnicas mencionadas en el presente documento, no pretenden formar conjuntos inflexibles de estándares, más bien, intenta servir de guía en el desarrollo de un estándar para este proyecto específico de software o para un conjunto de proyectos con similares características.

Propósito

El propósito de este documento es proporcionar estándares de codificación de estilo para el desarrollo de código fuente escrito en C #. La adhesión a una norma de estilo de codificación es un producto reconocido y las mejores prácticas para hacer más eficiente el desarrollo de equipos y mantenimiento de las aplicaciones más rentables. Aunque no es exhaustiva, estas directrices representan el nivel mínimo de normalización previsto en el código fuente de los proyectos escritos en C #.

Alcance

Este documento proporciona orientación sobre el formato, de comentarios, de nombres, y el estilo de código fuente de programación C # y es aplicable a bibliotecas de componentes, servicios web, sitios web y aplicaciones de cliente enriquecido.

Convenciones de Nombres

Nombrar proyectos, archivos de código fuente, e identificadores incluyendo campos, variables, propiedades, métodos, parámetros, clases, interfaces y espacios de nombres (Namespaces).

Pautas Generales

- Evitar nombres totalmente en MAYÚSCULAS o en minúsculas a menos que estos sean de una sola palabra.
- Nunca usar nombres que comiencen con caracteres numéricos.
- Todos los nombres a utilizar deben ser específicos y deben tener el largo necesario para ser entendidos.
- Evitar el uso de abreviaturas a menos que el nombre completo sea excesivo. En los casos necesarios, las abreviaturas deben ser totalmente conocidas y aprobadas por todos los integrantes.
- Nunca usar palabras reservadas para nombres.

Formato

Diseño de Clases

Las clases deben ser organizadas en regiones dentro de una aplicación utilizando un diseño determinado por el arquitecto de aplicaciones. Estos se pueden basar en la accesibilidad, tipo, o la funcionalidad.

Ejemplo:

```
// Diseño de clase basado en la accesibilidad
class Purchasing
{
    #region Main

    #region Public

    #region Internal

    #region Protected

    #region Private

    #region Extern

    #region Diseñador código generado
}
```

Nota: Utilice el mismo diseño coherente en todas las clases en una aplicación. Omitir o regiones si no se necesitan los elementos de la clase asociada.

Indicación de Alcance

Indique el alcance cuando se accede a todos los miembros de la clase estáticos y no estáticos. Esto proporciona una indicación muy clara de la utilización prevista del miembro. VisualStudio.NET intellisense se invoca automáticamente cuando se utiliza esta práctica, que proporciona una lista de todos los miembros de la clase disponibles.

Ejemplo:

```
string connectionString = DataAccess.DefaultConnectionString;  
float amount = this.CurrentAmount;  
this.discountedAmount = this.CalculateDiscountedAmount( amount, this.PurchaseMethod );
```

Nota: Incluir la palabra clave `this` ante todos los campos miembros, propiedades y métodos.

Incluir el nombre de la clase antes de que todos los campos estáticos, constantes, campos y métodos.

Comentarios

Intellisense Comentarios

Use la triple slash `"/ / /"` comentarios para documentar la interfaz pública de cada clase. Esto permitirá que Visual Studio.Net para recoger la información del método de Intellisense. Estos comentarios son necesarios antes de cada miembro de la clase pública, interna y protegida y opcional para los miembros privados.

End-Of-Line Comentarios

Utilice comentarios de fin de línea sólo con declaraciones de campos y variables miembro. Los utilizan para documentar el propósito de la variable que se está declarada.

Ejemplo:

```
private string name = string.Empty; // Name of control (defaults to blank)
```

Línea Individual Comentarios

Use los comentarios de una sola línea por encima de cada bloque de código relativas a una tarea concreta dentro de un método que realiza una operación importante o cuando se alcanza una condición significativa. Comentarios siempre deben comenzar con dos barras, seguido de un espacio.

Ejemplo:

```
// Compute total price including all taxes
float stateSalesTax = this.CalculateStateSalesTax( amount, Customer.State
);
float citySalesTax = this.CalculateCitySalesTax( amount, Customer.City );
float localSalesTax = this.CalculateLocalSalesTax( amount,
Customer.Zipcode );
float totalPrice = amount + stateSalesTax + citySalesTax +
localSalesTax;
Console.WriteLine( "Total Price: {0}", totalPrice );
```

//TODO: Comentarios

Use **//TODO:** Comentario para marcar una sección de código que necesita más trabajo antes de la liberación. El código fuente debe ser buscado estos comentarios antes de construir cada versión.

C-Style Comentarios

Utilice c-estilo `/* ... */` comentarios sólo para bloquear temporalmente grandes secciones de código durante el desarrollo y depuración. Código no se debe comprobar con estas secciones comentadas. Si el código ya no es necesario, eliminarlo. Aproveche las herramientas de control de código fuente para ver los cambios y supresiones de las versiones anteriores del código. Si el código debe ser registrado con grandes secciones comentadas, incluyen a **//TODO:** comentario sobre el bloque comentado describir por qué se registró esa manera.

Capitalización y nomenclatura

Capitalización

Siga las Instrucciones de nomenclatura estándar establecidas por el equipo NET Framework utilizando sólo tres estilos de mayúsculas: Pascal, Camel, y la cubierta superior.

Ejemplos:

Tipo de Identificador	Estilo de Capitalización	Ejemplo (s)
Abbreviations	Upper	ID, REF
Namespaces	Pascal	AppDomain, System.IO
Classes & Structs	Pascal	AppView
Constants & Enums	Pascal	TextStyles
Interfaces	Pascal	IEditableObject
Enum values	Pascal	TextStyles.BoldText
Property	Pascal	BackColor
Variables, and Attributes	Pascal (public)	WindowSize

	Camel (private, protected, local)	windowWidth, windowHeight
Methods	Pascal (public, private, protected)	ToString()
	Camel (parameters)	SetFilter(string filterValue)
Local Variables	Camel	recordCount

Nota: En la nomenclatura Pascal, la primera letra del identificador se escribe con mayúscula, así como la primera letra de cada palabra concatenada. Este estilo se utiliza para todos los identificadores públicos dentro de una biblioteca de clases, incluyendo espacios de nombres, clases y estructuras, propiedades y métodos.

En la nomenclatura del Camello, la primera letra del identificador está en minúscula, pero la primera letra de cada palabra concatenada se escribe con mayúscula. Este estilo se utiliza para los identificadores privados y protegidos dentro de la biblioteca de clases, parámetros que se pasan a los métodos y las variables locales dentro de un método.

Caja superior o sólo se utiliza para los identificadores abreviados y siglas de las cuatro letras o menos.

Naming

En caso de nombrar Clases, Interfaces y Namespace se utilizará la nomenclatura Pascal y para el caso de declaraciones de variables se utilizará la notación Camello.

Las pautas que establecen cada nomenclatura se dejaron expuestas en el punto anterior.

Programación

Namespaces

Los espacios de nombres representan el embalaje lógica de capas de componentes y subsistemas. La plantilla de declaración de espacios de nombres es: `CompanyName.ProjectOrDomainName.PackageName.SubsystemName`.

Ejemplos:

```
Microsoft.Data.DataAccess
Microsoft.Logging.Listeners
```

Nota: De proyectos y espacios de nombres de nivel de paquete normalmente serán predeterminados por un arquitecto de aplicaciones para cada proyecto.

Uso de nomenclatura Pascal al nombrar a los espacios de nombres del subsistema.

Clases y estructuras

Clases y estructuras representan los "sustantivos" de un sistema. Como tales, deben ser declaradas usando la siguiente plantilla: Sustantivo + Qualifier (s). Las Clases y estructuras deberán declaradas con calificativos que reflejan su derivación de una clase base siempre que sea posible.

Ejemplos:

```
CustomerForm : Form
CustomerCollection : CollectionBase
```

Notas: Uso de la nomenclatura Pascal al nombrar las clases y estructuras.

Las Clases y estructuras deberán ser divididas en distintas regiones como se describió previamente en las directrices de diseño de clase.

Todas las clases públicas y sus métodos deben ser documentadas mediante el Intellisense de triple slash `"/ / /"`, comentarios integrados en Visual Studio.Net. Use este estilo de comentario para documentar los efectos de la clase y sus métodos.

Los valores por defecto para los campos deben ser asignados en la línea donde se declara el campo. Estos valores son asignados en tiempo de ejecución sólo antes de llamar al constructor. Esto evita que el código de valores por defecto en un solo lugar, sobre todo cuando una clase contiene varios constructores.

Interfaces

Interfaces expresan contratos de comportamiento que las clases derivadas deben implementar. Los nombres de interfaces deben utilizar los sustantivos, sintagmas nominales o adjetivos que expresan claramente el comportamiento que declaren.

Ejemplos:

```
IComponent
IFormattable
ITaxableProduct
```

Notas: Los nombres de interfaz de prefijo con la letra 'I'.

Usar la nomenclatura Pascal al nombrar interfaces.

Constantes

Constantes y variables de sólo lectura estáticos deben ser declaradas usando la siguiente plantilla: Adjetivo (s) + sustantivo + Qualifier (s)

Ejemplo:

```
public const int DefaultValue = 25;
public static readonly string DefaultDatabaseName = "Membership";
```

Nota: Uso de la nomenclatura Pascal al nombrar constantes y variables estáticas sólo lectura.

Priorizar el uso de `static readonly` sobre `const` para las constantes públicas siempre que sea posible. Las Constantes declaradas utilizando `const` se sustituye en el código para acceder a ellos en tiempo de compilación. Utilización de variables de sólo lectura estáticos garantiza que se accede a los valores constantes en tiempo de ejecución. Esto es más seguro y menos propenso a la rotura, especialmente cuando se accede a un valor constante de un conjunto diferente.

Las enumeraciones

Las enumeraciones deben ser declaradas usando la siguiente plantilla: Adjetivo (s) + sustantivo + Qualifier (s)

Ejemplo:

```
/// <summary>
/// Enumerates the ways a customer may purchase goods.
/// </summary>
[Flags]
public enum PurchaseMethod
{
    All          = ~0,
    None         = 0,
    Cash         = 1,
    Check        = 2,
    CreditCard   = 4,
    DebitCard    = 8,
    Voucher      = 16,
}
```

Notas: Uso de la nomenclatura Pascal al nombrar las enumeraciones.

Utilice el botón [Banderas] atributo sólo para indicar que la enumeración puede ser tratada como un campo de bits, es decir, un conjunto de indicadores.

Atributos

Variables, campos y parámetros deben ser declaradas usando la siguiente plantilla: Adjetivo (s) + sustantivo + Qualifier (s)

Ejemplos:

```
private int lowestCommonDenominator;  
private float firstRedBallPrice;
```

Nota:

Utilice la nomenclatura Camel al nombrar los atributos.

Propiedades

Las propiedades deben ser declaradas usando la siguiente plantilla: Adjetivo (s) + sustantivo + Qualifier (s)

Ejemplos:

```
public TotalPrice  
{  
    get  
    {  
        return this.totalPrice;  
    }  
    set  
    {  
        // Set value and fire changed event if new value is different  
        if( !object.Equals( value, this.totalPrice ) )  
        {  
            this.totalPrice = value;  
            this.OnTotalPriceChanged();  
        }  
    }  
}
```

Nota:

Las propiedades deben llevar la misma denominación que los atributos que manejan, se debe utilizar la nomenclatura Pascal.

Métodos

Los métodos deben ser nombrados con el siguiente formato: Verbo + adjetivo (s) + sustantivo + Qualifier (s)

Ejemplo:

```
private Ball FindRedCansByPrice(  
    float price,  
    ref int canListToPopulate,  
    out int numberOfCansFound )
```

Nota:

Los parámetros deben ser agrupados por su mutabilidad (de menos a más mutable) como se muestra en el ejemplo anterior.

Si es posible, evitar los métodos de sus centros que salir. Un método bien escrito sólo debe salir de un punto: en su extremo.

Evite métodos de gran tamaño. Mientras el cuerpo de un método se acerca de 20 a 30 líneas de código, busque bloques que pueden ser divididos en sus propios métodos y posiblemente compartida por otros métodos.

Si usted se encuentra con el mismo bloque de código más de una vez, es un buen candidato para un procedimiento separado.

Grupo como métodos dentro de una clase juntos en una región y ordenarlos según la frecuencia de uso (es decir, con mayor frecuencia llamados métodos deben estar cerca de la parte superior de sus regiones).

Variables y parámetros

Las variables y parámetros deben ser nombrados siguiendo la notación Carmel, el nombre debe estar presidido por una sigla identificadora del tipo de dato al que pertenece

La siguiente tabla muestra el listado de tipos de datos con sus respectivas siglas:

i	Entero (int, Int32)
n	Enumerador (enum).
t	Estructura o tipo (struct).
e	Evento (event). (para Callbacks)
o	Objeto (object).
d	Doble o decimal (double).
c	Carácter (char)
b	Booleano (bool)
l	Entero largo (long, Int64).
f	Flotante (float).
s	Simple o single (short, Int16).
p	Parámetro (utilizado en métodos y/o funciones).
sz	Cadena (string).

Por ejemplo:

```
private int iDni;
```

```
private string szNombre;
```

Controladores de eventos

Los controladores de eventos se deben declarar con el siguiente formato: `ObjectName_EventName`

Ejemplo:

```
private HelpButton_Click( object sender, EventArgs e )
```

Control de errores

Utilice excepciones sólo en casos excepcionales, no para el flujo del programa de rutina. Las excepciones tienen sobrecarga de rendimiento significativa.

Nota:

Pasar una cadena descriptiva al constructor cuando se produzca una excepción.

Utilizar mensajes de error gramaticalmente correctas, incluyendo puntuación final. Cada frase de la cadena de descripción de una excepción debe terminar con un punto.

Si una propiedad o método produce una excepción en algunos casos, documentar esto en los comentarios para el método. Incluya qué excepción se produce y lo que hace que se lance.

- Ejemplo: Comentario de la propiedad `Order.TotalCost` puede leer "Obtiene o establece el costo total de una orden si la propiedad `CostoTotal` se establece cuando se debe calcular el costo, una excepción `InvalidOperationException`".

- Utilizar las siguientes excepciones en su caso:

- `ArgumentException` (y `ArgumentNullException`, `ArgumentOutOfRangeException`, `IndexOutOfRangeException`): Se utiliza en la comprobación de los parámetros de entrada válidos a método.

- `InvalidOperationException`: Se utiliza cuando una llamada al método no es válida para el estado actual de un objeto.

Ejemplo: `CostoTotal` no se puede establecer si el costo debe ser calculado. Si la propiedad se establece y no la regla, una excepción `InvalidOperationException`.

- `NotSupportedException`: Se utiliza cuando una llamada al método no es válido para la clase.

Ejemplo: cantidad, una lectura / escritura de la propiedad virtual, es reemplazado por una clase derivada. En la clase derivada, la propiedad es de sólo lectura. Si la propiedad se establece, una `NotSupportedException`.

- `NotImplementedException`: Se utiliza cuando un método no está implementado para a clase actual.

Ejemplo: Un método de interfaz se apagó y aún no implementadas. Este método debe lanzar una NotImplementedException.

Derivar sus propias clases de excepción durante escenarios de programación. Todas las nuevas excepciones derivadas deben basarse en la clase Exception núcleo.

Ejemplo: DeletedByAnotherUserException: Excepción. Lanzado para indicar un registro que está siendo modificado ha sido eliminado por otro usuario.

Rethrow atrapado excepciones correctamente.

En el siguiente ejemplo se produce una excepción detectada y relanzada incorrectamente:

```
catch( Exception ex )
{
    LogManager.Publish( ex );
    throw ex;    // INCORRECT - we lose the call stack of the exception
}
```

Registramos todas las excepciones no controladas en nuestras aplicaciones, pero a veces puede lanzar de nuevo para permitir que los sistemas de nivel superior determinar cómo proceder. El problema viene con la banda - que funciona mucho mejor para hacer esto:

```
catch( Exception ex )
{
    LogManager.Publish( ex );
    throw;    // CORRECT - rethrows the exception we just caught
}
```

Nótese la ausencia de un argumento para la instrucción throw en la segunda variante.

La diferencia entre estas dos variaciones es sutil pero importante. En el primer ejemplo, la persona que llama nivel superior no va a obtener toda la información sobre el error original. La pila de llamadas de excepción se sustituye por una nueva pila de llamadas que se origina en el "throw ex" declaración - que no es lo que queremos grabar. El segundo ejemplo es el único que en realidad re-produce la excepción original, preservando el seguimiento de la pila donde se produjo el error original.

Nombres de Controles

Para controles de formularios se utilizara Notación Húngara que Consiste en prefijos en minúsculas que se añaden a los nombres de las variables (controles en este caso), y que indican su tipo. El resto del nombre indica, lo más claramente posible, la función que realiza la variable, formateado en PascalCase.

Evite usar más de 25 caracteres y "_" (guiones bajos) en los nombres de controles.

En la siguiente tabla se muestra un listado de los prefijos a utilizar:

Prefijo	TipoPrefijo	Ejemplo
btn	Button	btnAceptar
chk	CheckBox	chkEliminar
ctl	Control	ctlControlPersonalizado

dgv	DataGridView	dgvListadoCliente
frm	Form	frmActualizarClientes
lbl	Label	lblTitulo
lst	ListBox	lstRubros
mnu	Menu	mnuActualizarClientes
img	Image	imgFotoCliente
dt	DataTable	dtPersona
ds	DataSet	dsFacturacion
tmr	Timer	tmrFecha
dtr	DataRow	dtrCliente
dtp	DateTimePicker	dtpFecha
cal	Calendar	calCalendario
cmb	ComboBox	cmbFormasDePago
opt	RadioButton	optActivo
txt	TextBox	txtApellidoNombre

Namespace

Los Namespaces son utilizados para crear un sistema destinado organizar la forma de exponer la funcionalidad de un aplicativo. Dicha funcionalidad se expresa en clases .NET.

El esquema de Namespaces es jerárquico, es decir permite definir un sistema de nombres comenzando por un mayor nivel de abstracción hasta llegar a niveles más específicos.

A continuación se visualiza la regla general para nombrar Namespaces:

<Compañía>.<[Tecnología]/[Sistema]>[.<Módulo>][.Diseño]

Dado que los aplicativos se encontraran basados en un sistema de múltiples capas, inclusive capas físicamente bien diferenciadas, la regla que utilizaremos para definir Namespaces es:

<Compañía>.<[Tecnología][Sistema]>[.Capa Física][.< Módulo >][.Diseño]

A continuación se citan algunos ejemplos:

Apu.Tools

Apu.Framework.Bases.BackEnd

Apu.SistemaFacturacion

Apu.SistemaFacturacion.Frontend

Apu.SistemaFacturacion.Frontend.FrontController

Apu.SistemaFacturacion.Backend

Apu SistemaFacturacion.Backend.Services

Como se puede visualizar en los ejemplos, de acuerdo al área de competencia de la clase, se utiliza un namespace de mayor o menor nivel de abstracción o detalle. Es decir, si existen clases de tipo utilitarios que pueden ser utilizadas por más de un aplicativo, se utilizará el namespace `Sicom.Tools` o `Sicom.Framework`. Mientras que si el utilitario solo puede ser utilizado dentro del Sistema de Facturación, el namespace a utilizar sería `sistema SistemaFacturacion.NombreSistema.Tools`.

En los ejemplos también podemos visualizar la existencia de dos capas físicas bien diferenciadas: Frontend y Backend. Existe una nomenclatura que permite definir el namespaces para clases que son compartidas entre ambas capas:

“Common”. Un ejemplo de cuando utilizar Common es cuando se definen los Request del Framework de `Sicom.Framework`, se utilizan tanto en el Frontend como en el Backend.

De Frontend se desprenden los namespaces utilizados para componentes como:

Páginas ASPX, UIComponents, Piezas propias del Front Controller, Etc.

De Backend se desprenden los namespaces utilizados para representar a todos los servicios y las capas lógicas que se encuentran detrás: Componentes de Negocio, Componentes de Soporte, Entidades de Negocio, Entidades de Soporte, Etc.